

Nama : Muhammad Alfin Maulana

NPM : 5230311052

Catatan BAB VIII CODEIGNITER (CI)

Pengenalan CI

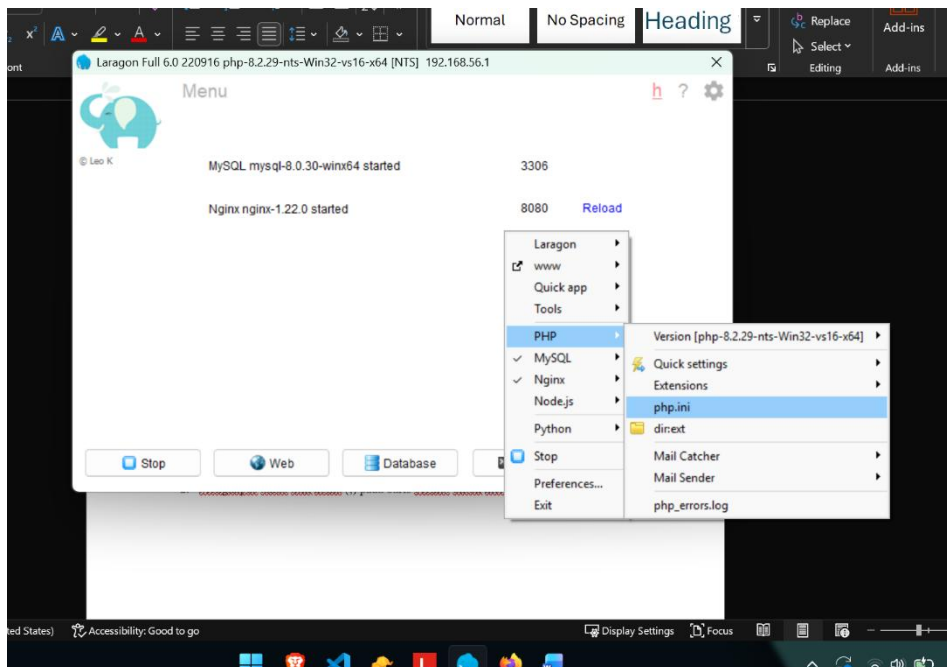
Pada bab ini, saya mengeksplorasi CodeIgniter 4 (CI4), sebuah framework PHP yang dikenal *lightweight* namun handal. Berbeda dengan pendekatan prosedural, CI4 memaksa kita menggunakan pola arsitektur MVC (Model-View-Controller). Fokus praktikum ini adalah instalasi lingkungan kerja, konfigurasi database menggunakan teknik *Migrations*, serta pemahaman alur dasar *Routing* ke *Controller*.

Konfigurasi Lingkungan (Environment Setup)

Sebelum menyentuh kodingan, hal krusial yang sering menjadi penghalang instalasi adalah konfigurasi ekstensi PHP. CodeIgniter 4 membutuhkan dependensi intl dan mbstring agar dapat diinstal via Composer.

Langkah Perbaikan php.ini:

1. Saya membuka file php.ini di dalam folder konfigurasi PHP (biasanya di xampp/php/php.ini).



2. Menghapus tanda titik koma (;) pada baris berikut untuk mengaktifkan ekstensi:

`extension=intl`

`extension=mbstring`

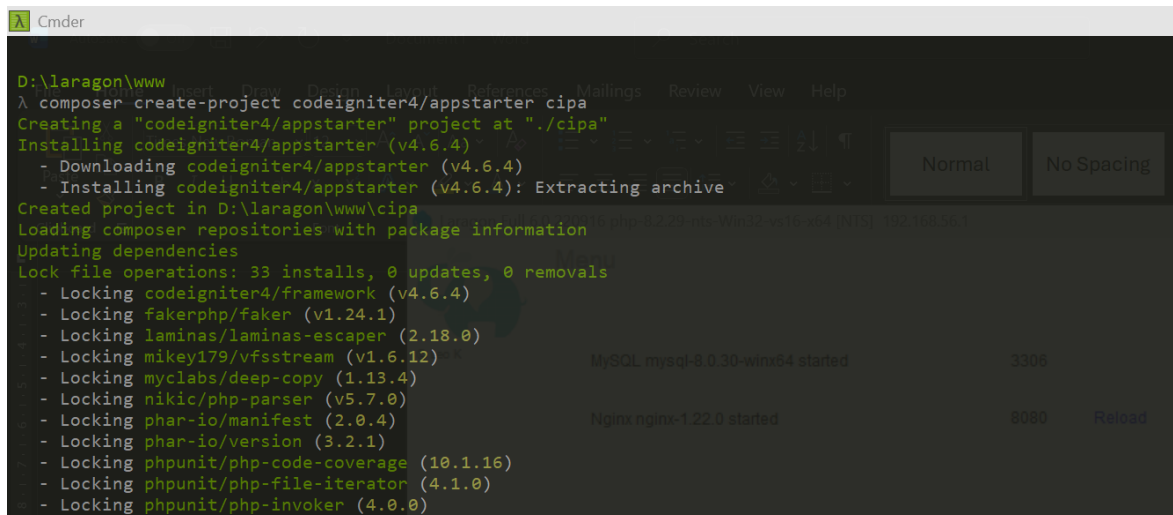
```
933 extension=fileinfo
934 extension=gd
935 ;extension=gettext
936 ;extension=gmp
937 extension=intl
938 ;extension=imap
939 extension=mbstring
940 extension=exif ; Must be after mbstring as it depends on it
941 extension=mysqli
942 ;extension=oci8_12c ; Use with Oracle Database 12c Instant Client
943 ;extension=oci8_19 ; Use with Oracle Database 19 Instant Client
944 ;extension=odbc
945 extension=openssl
946 ;extension=pdo_firebird
```

Langkah ini wajib dilakukan karena library internal CI4 memerlukan ekstensi internasionalisasi (intl).

Instalasi Project via Composer

Saya memilih menggunakan Composer untuk instalasi karena memudahkan manajemen dependensi dan update versi di masa depan.

Perintah CLI:



```
Cmder
D:\laragon\www
λ composer create-project codeigniter4/appstarter cipa
Creating a "codeigniter4/appstarter" project at "./cipa"
Installing codeigniter4/appstarter (v4.6.4)
- Downloading codeigniter4/appstarter (v4.6.4)
- Installing codeigniter4/appstarter (v4.6.4): Extracting archive
Created project in D:\laragon\www\cipa
Loading composer repositories with package information
Updating dependencies
Lock file operations: 33 installs, 0 updates, 0 removals
- Locking codeigniter4/framework (v4.6.4)
- Locking fakerphp/faker (v1.24.1)
- Locking laminas/laminas-escaper (2.18.0)
- Locking mikey179/vfsstream (v1.6.12)
- Locking myclabs/deep-copy (1.13.4)
- Locking nikic/php-parser (v5.7.0)
- Locking phar-io/manifest (2.0.4)
- Locking phar-io/version (3.2.1)
- Locking phpunit/php-code-coverage (10.1.16)
- Locking phpunit/php-file-iterator (4.1.0)
- Locking phpunit/php-invoker (4.0.0)
```

Saya menamai folder proyek ini cipa. Saat proses ini, jika versi PHP di lokal tidak memenuhi syarat (misal < 8.1 untuk CI4 versi terbaru), Composer akan memberikan pesan error. Solusinya adalah upgrade XAMPP atau mendowngrade versi CI4 yang diminta.

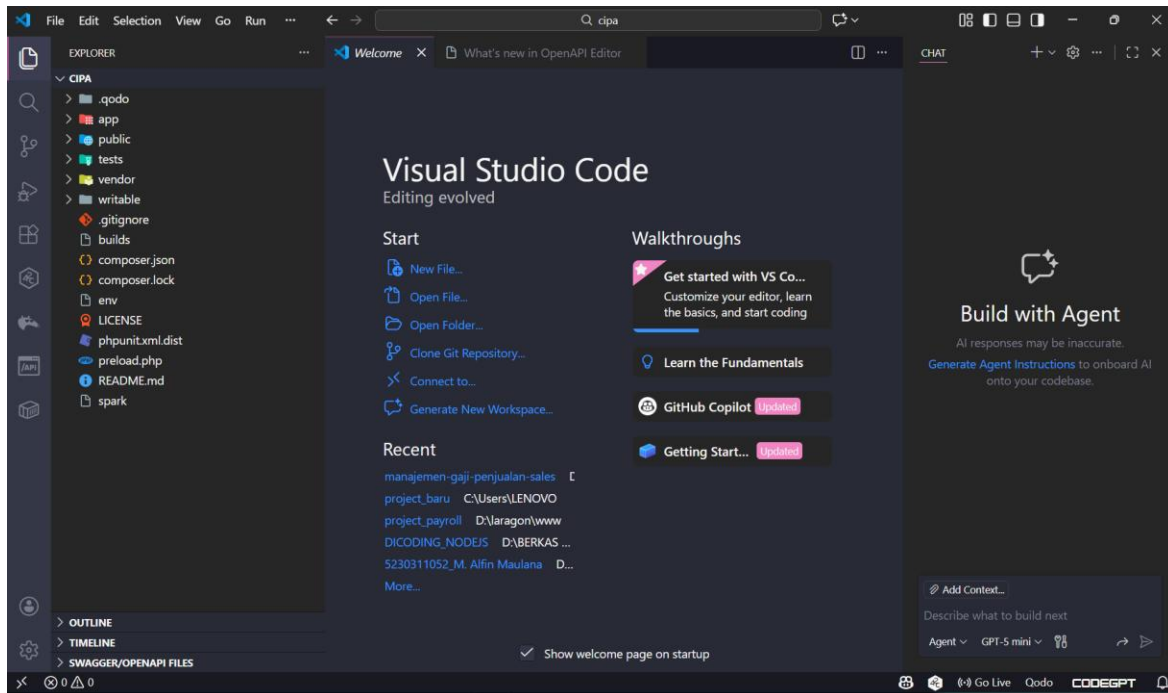
Kita bisa masuk ke dalam project yang telah dibuat dengan memerintahkan perintah di bawah ini

```
No security vulnerability advisories found

D:\laragon\www
λ cd cipa\

D:\laragon\www\cipa
λ code .

D:\laragon\www\cipa
λ
```

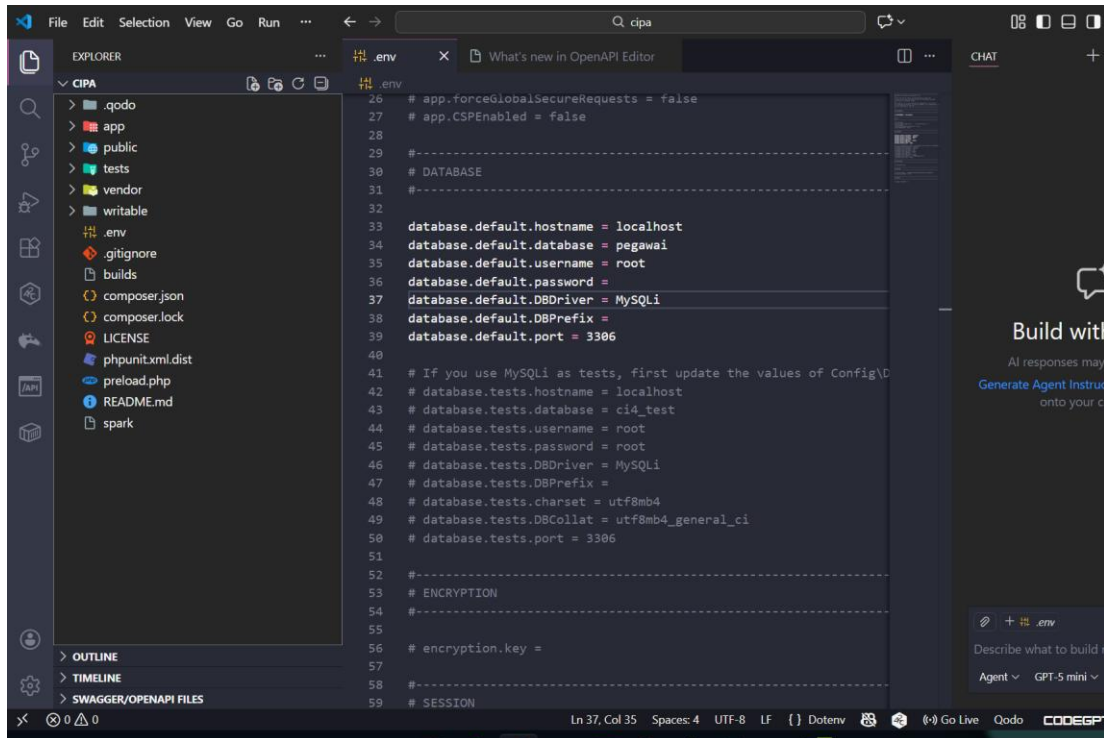


Bisa dilihat di atas project CI yang telah dibuat berhasil dibuat, untuk selanjutnya saya akan menjelaskan lebih mendalam apa saja fitur yang ada di dalam framework CodeIntegner.

Konfigurasi Database & Environment (.env)

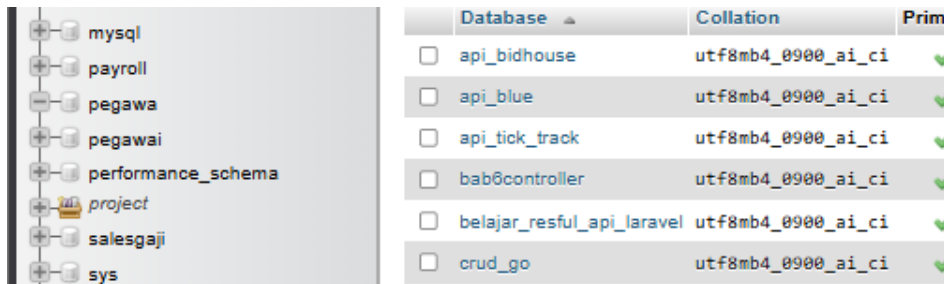
Salah satu praktik keamanan terbaik di CI4 adalah tidak menaruh kredensial database langsung di file PHP, melainkan di file environment variable (.env).

1. **Setup File .env:** Saya menyalin file env bawaan menjadi .env, lalu melakukan konfigurasi berikut untuk koneksi database MySQL:



Mengaktifkan mode development untuk melihat detail error saat debugging.

2. Persiapan Database: Membuat database kosong bernama pegawai melalui phpMyAdmin.



Setelah saya menyesuaikan isi .env untuk menyelaraskan dengan database yang telah dibuat, disini saya membuat sebuah database langsung di dalam phpMyAdmin dengan nama pegawai.

Database Migrations (Praktik Modern)

Alih-alih membuat tabel secara manual (GUI) yang sulit dilacak perubahannya, saya menggunakan fitur Migrations. Ini memungkinkan skema database ditulis dalam kode PHP.

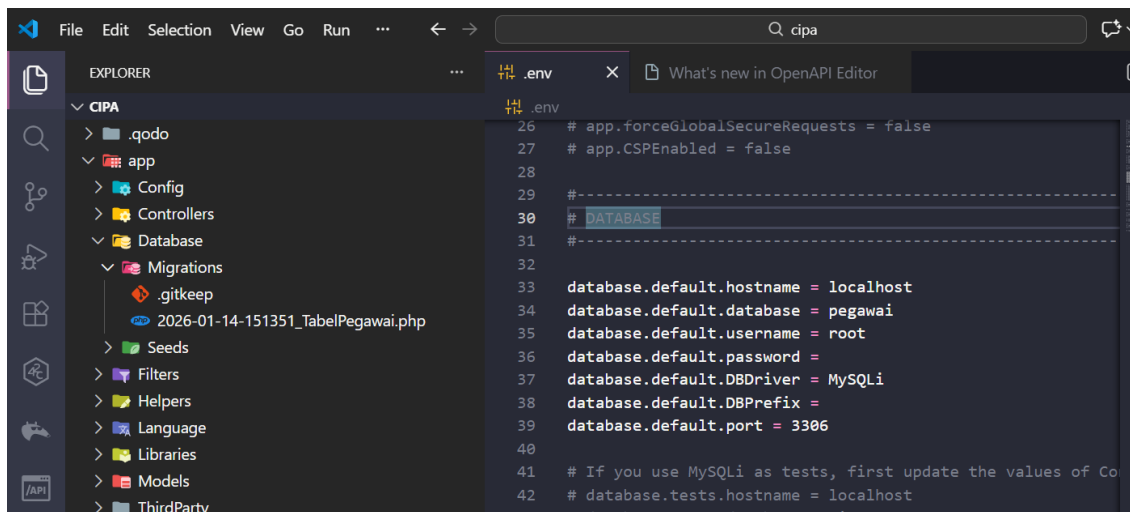
1. Membuat File Migrasi: Saya menggunakan *Spark* (CLI tool bawaan CI4) untuk generate file:

```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 /d/laragon/www/cipa
● $ php spark make:migration tabel_pegawai

CodeIgniter v4.6.4 Command Line Tool - Server Time: 2026-01-14 15:13:50 UTC+00:00

File created: APPPATH\Database\Migrations\2026-01-14-151351_TabelPegawai.php
```

Nah perintah di atas merupakan perintah untuk membuat sebuah table dengan database yang telah dibuat secara langsung GUI phpMyAdmin. Jika sudah berhasil dibuat kita bisa melihat di dalam folder app\Migrations



Bisa dilihat table_pegawai berhasil dibuat dan selanjutnya saya akan membuat sebuah schema table di dalam table yang telah dibuat sebelumnya.

2. **Mendefinisikan Schema Tabel:** Dalam file migrasi tersebut, saya mendefinisikan struktur tabel pegawai pada method up(). *(Catatan: Saya memperbaiki beberapa typo penulisan dari modul referensi agar kode berjalan lancar).*

```
<?php

namespace App\Database\Migrations;
use CodeIgniter\Database\Migration;

class TabelPegawai extends Migration
{
    public function up()
    {
        $this->forge->addField([
```

```

        'id' => [
            'type'          => 'INT',
            'constraint'    => 5,
            'unsigned'      => true,
            'auto_increment' => true,
        ],
        'nama' => [
            'type'          => 'VARCHAR',
            'constraint'    => '50',
        ],
        'alamat' => [
            'type' => 'TEXT',
            'null' => true,
        ],
    ],
]);

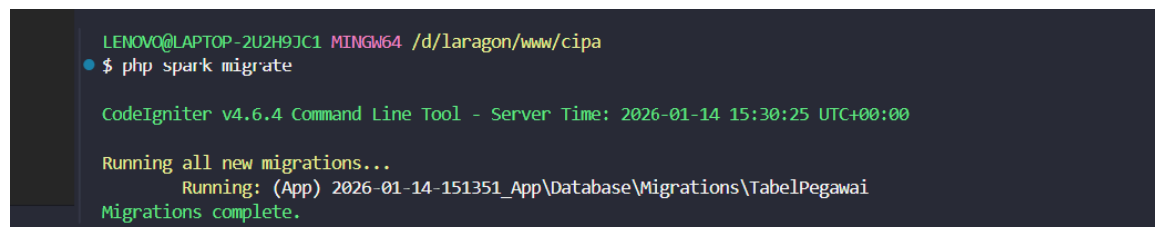
$this->forge->addKey('id', true);
$this->forge->createTable('pegawai');
}

public function down()
{
    $this->forge->dropTable('pegawai');
}
}

```

Setelah membuat sebuah schema table sederhana, selanjutnya saya akan menjalankan sebuah perintah untuk menjalankan schema table yang telah dibuat.

3. Eksekusi Migrasi:



```

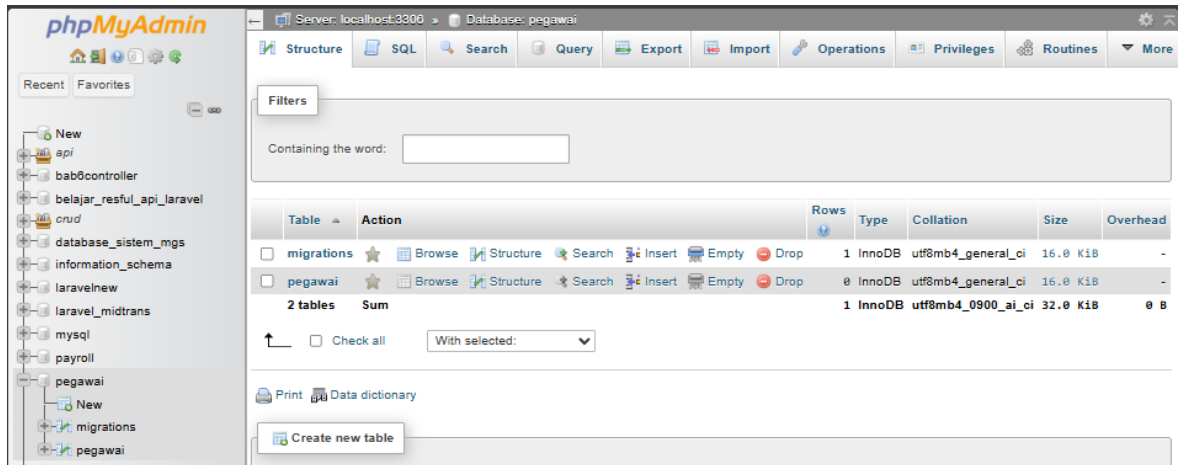
LENOVO@LAPTOP-2U2H9JC1 MINGW64 /d/laragon/www/cipa
$ php spark migrate

CodeIgniter v4.6.4 Command Line Tool - Server Time: 2026-01-14 15:30:25 UTC+00:00

Running all new migrations...
Running: (App) 2026-01-14-151351_App\Database\Migrations\TabelPegawai
Migrations complete.

```

Perintah ini mengonversi kode PHP di atas menjadi query SQL CREATE TABLE secara otomatis.



Jika dilihat di dalam phpMyAdmin di dalam database yang telah kita buat sudah ada table pegawai yang dibuat di dalam perintah terminal tadi.

Implementasi Logic: Controller & Routing

Setelah database siap, saya membuat logika backend sederhana.

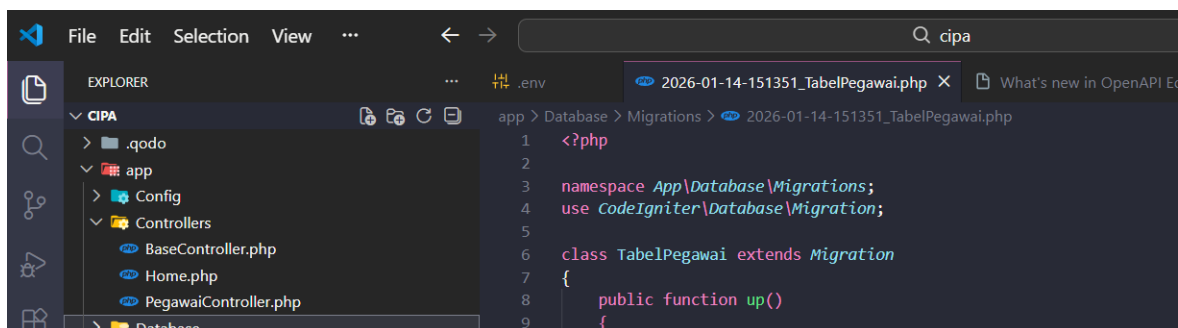
1. **Membuat Controller:** Saya men-generate controller baru agar kode lebih terorganisir.

```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 /d/laragon/www/cipa
$ php spark make:controller PegawaiController

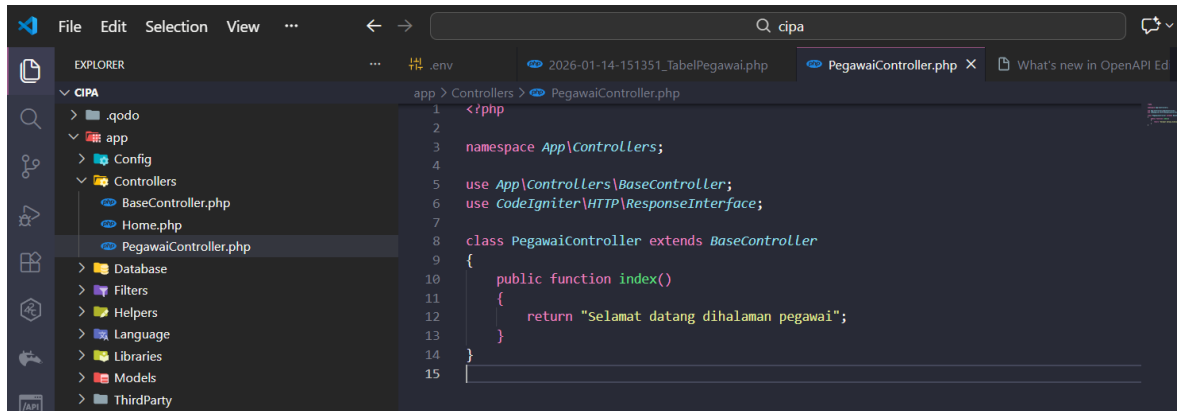
CodeIgniter v4.6.4 Command Line Tool - Server Time: 2026-01-14 15:39:42 UTC+00:00

File created: APPPATH\Controllers\PegawaiController.php
```

Perintah di atas adalah perintah untuk membuat sebuah controller, kurang lebih fungsinya untuk membuat sebuah logic code di dalam framework CI ini, penggunaanya sama seperti di dalam framework Laravel.



2. **Menambahkan Logika Sederhana:** Di file app/Controllers/PegawaiController.php, saya menambahkan fungsi index yang mengembalikan respon string.



```
1 <?php
2
3 namespace App\Controllers;
4
5 use App\Controllers\BaseController;
6 use CodeIgniter\HTTP\ResponseInterface;
7
8 class PegawaiController extends BaseController
9 {
10     public function index()
11     {
12         return "Selamat datang di halaman pegawai";
13     }
14 }
15
```

Controller di atas berfungsi untuk membuat/menampilkan sebuah teks yang bertuliskan seperti di atas.

3. **Mendaftarkan Route:** Agar fungsi di atas bisa diakses user, saya memetakan URL /pegawai ke controller tersebut pada file app/Config/Routes.php.



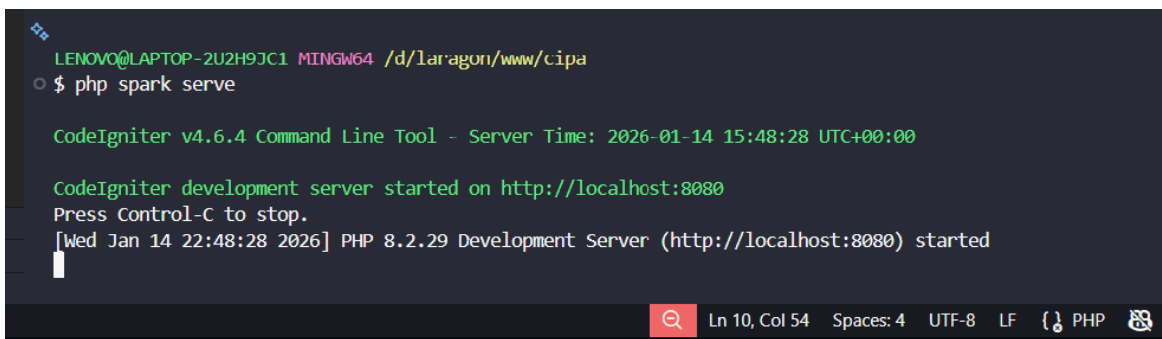
```
1 <?php
2
3 use CodeIgniter\Router\RouteCollection;
4
5 /**
6  * @var RouteCollection $routes
7  */
8 $routes->get('/', 'Home::index');
9
10 $routes->get('/pegawai', 'PegawaiController::index');
```

Nah isi file controller yang telah kita buat dengan perintah di atas, untuk memanggil controller yang telah dibuat melalui route ini.

Uji Coba (Deployment Lokal)

Langkah terakhir adalah menjalankan *development server*.

1. **Start Server:**



```
LENOVO@LAPTOP-2U2H9JC1 MINGW64 /d/laragon/www/cipa
$ php spark serve

CodeIgniter v4.6.4 Command Line Tool - Server Time: 2026-01-14 15:48:28 UTC+00:00

CodeIgniter development server started on http://localhost:8080
Press Control-C to stop.
[Wed Jan 14 22:48:28 2026] PHP 8.2.29 Development Server (http://localhost:8080) started
```


Terminal akan menampilkan bahwa server berjalan di <http://localhost:8080>.

2. **Hasil Akhir:** Saat saya mengakses <http://localhost:8080/pegawai> di browser, muncul teks: **"Selamat datang di halaman pegawai"**.



Praktikum ini menegaskan perbedaan alur kerja framework modern dibandingkan PHP Native. Penggunaan **CLI (Spark)** sangat mempercepat produktivitas—seperti membuat file migrasi dan controller tanpa perlu copy-paste file manual. Selain itu, penggunaan **Migrations** menjamin konsistensi struktur database, yang sangat penting jika saya bekerja dalam tim backend di masa depan.